
An Investigation into Privacy Methods for Current Prediction Using Deep Learning Models

Matthew Jones, Aaryan Raghuvanshi, & Ren Yanghui

Oregon State University, School of Electrical Engineering and Computer Science
jonesm25, raghuva, & renya@oregonstate.edu

Abstract

This project investigated two privacy methods that can be employed by a deep learning model used for current prediction to protect the confidentiality of the manufacturing process data. The goal was to see how these methods impacted model accuracy and training latency, and determine whether their success against standard privacy attacks was worth accepting that impact. Prior implementations of these methods (both attacks and defenses) were not used in the kind of current prediction model employed here. Our technical approach was to implement both Differential Privacy and Split Learning (in Python) and apply them to the model both individually and together, since they are not mutually exclusive. We then attacked the resulting models with Embedding Inversion, Edge Reconstruction, and Membership Inference. Our results show that that DP leads to a significant impact to accuracy and that the attack success is not significantly worse than the baseline.

1 Introduction

This project is based on the PhD research for one of the authors, Matthew Jones. His research focus is “optimizing analog integrated circuit layout silicon area by strategically violating physical design rules while ensuring overall chip reliability goals are still met.” Figure 1 shows the high-level system flowchart. Functionally, it’s three connected activities, but it is only the first one, the current prediction model (CPM), that is relevant here because it is the basis this project.

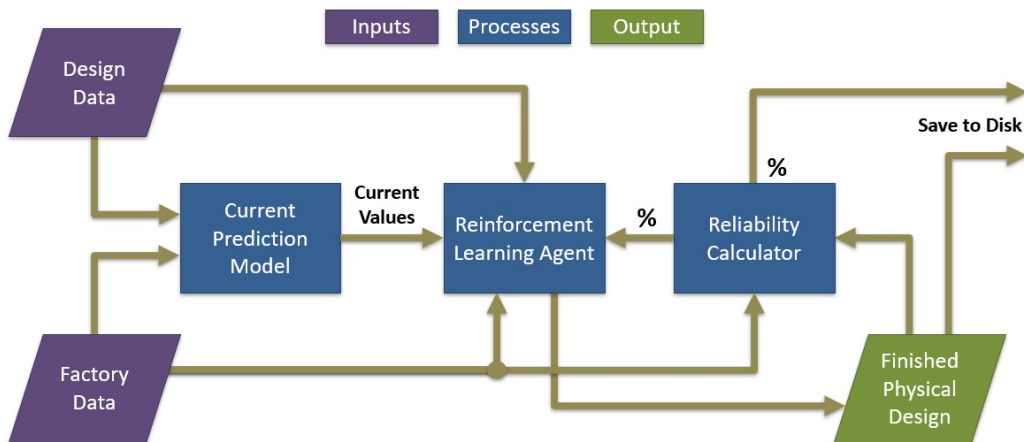


Figure 1: High-Level System Flowchart

The privacy concern for the CPM, which is a Graph Neural Network (GNN), is related to the sharing of Factory Data. These data are proprietary information for each foundry (an independent company that operates semiconductor factories) and they are reluctant to share it without strong non-disclosure agreements in place. Since the CPM is to be trained on a dataset created by simulations using the factory data, keeping the dataset private is a concern.

The remainder of this paper describes the Current Prediction Model (Section 2), including how the dataset was generated (Section 2.1) and the GNN itself (Section 2.2) to provide context; Related Work for our privacy strategies (Section 3); our Approach to applying those strategies (Section 4); Experimental Results comparing the strategies and attack methods (Section 5); and finally a Discussion and Conclusion (Section 6). The Appendix contains the slides from the Final Presentation (starting on page 21) and other supporting material.

2 Current Prediction Model

An alternative to circuit simulation is to predict the current using a deep learning model. The model can be trained on simpler simulations done in advance on the same manufacturing process, and the predictions can be made using the netlists of the sub-blocks. The predictions may not be as accurate as carefully done simulations, but they don't have to be for their intended purpose.

2.1 Generating Training Datasets

The first step toward training *any* type of neural network is to create a dataset, and the first step toward creating *this* dataset is to choose a test circuit. The test circuit is a Cascaded Inverter Chain. The circuit and its testbench are shown in Figure 2. The SPICE netlist is in the Appendix on page 16. [10] The target node, which represents the connection between two circuit blocks, is between them. Functionally, it is very simple circuit, but requires additional elements related to the testbench in order to capture the required current values in the simulation results necessary to generate the dataset.

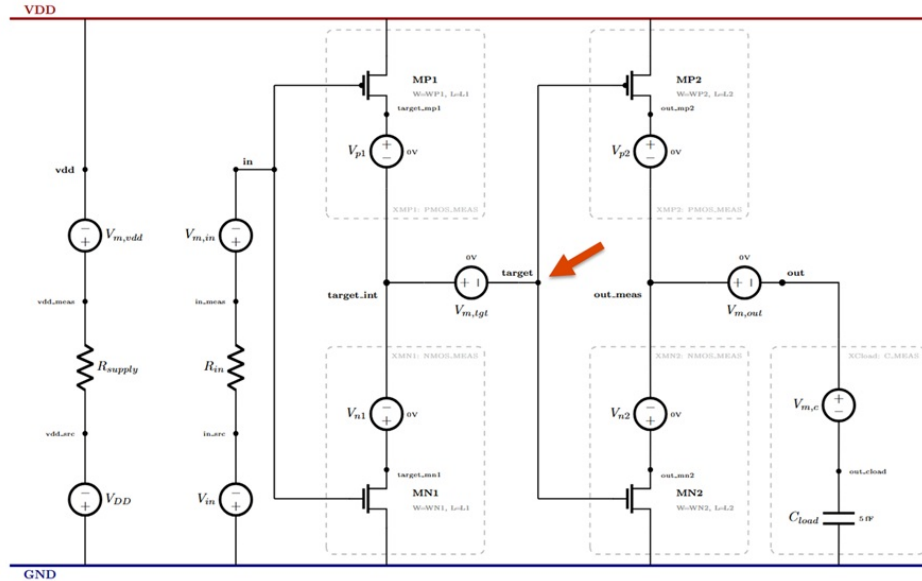


Figure 2: Testbench for Cascaded Inverter Chain

The testbench is comprised of transistors, subcircuits, measurement sources, and supply and input sources. The transistors are what make up the inverters. The rest of the elements are to facilitate measurement for data capture, specifically I_{target} , the current through the target node between the two inverters. A description of each element can be found in the Appendix on page 19. The dataset was generated by simulating, using *Ngspice*, the circuit testbench across different process models,

process corners and skews, and transistor sizes. [9] Each simulation represents a unique combination of process technology, operating conditions, device asymmetry, and circuit parameters. A description of these variations can also be found in the Appendix on page 20. Note how much had to be added to the original circuit *just* to measure the current. As previously mentioned, this extra work is part of the reason why a current prediction model is a practical solution.

The dataset was generated from simulations using a PTM model, 22nm_HP.pm (higher threshold voltage for reduced leakage and low power 22nm node, see page 17). [6] Since there are five process corners and five process skews there is a total of 25 combinations. The dataset contains 100,000 instances, so there are 4,000 different variations in the transistor sizes for each combination. Figure 3 shows a single instance of the dataset.

ID is the instance number. *Design* is the circuit. Even though it's the same for every instance, it makes sense to include this in the dataset because of future work will include multiple designs. *Size* and *Option* are the process. 22 is the minimum feature size in nanometers and HP means it's the High Performance version of the model. This is as opposed to LP, which stands for Low Power. *PVT* and *Skew* are simulation corners. PVT is Process, Voltage, and Temperature. The voltage is the supply voltage. Skew is how the transistors perform. SS means both N-type and P-type are slow. Skew can also be fast/fast, fast/slow, slow/fast, and typical/typical. The *Ws* and *Ls* are transistor sizes. For example, WN1 is the gate width of the N-type transistor in the first stage. Similarly for the others. It's typical for the N-type and P-type transistors to have the same gate length, so there are only two lengths. VDD is the supply voltage and Temp is the temperature. The rest are the voltages and current for the nodes, the last being *I_target*, the target current the model is going to predict.

```
{
  "ID": 0,
  "Design": "2inv",
  "Size": "22",
  "Option": "HP",
  "PVT": "typical",
  "Skew": "SS",
  "WN1": 3.987860998114745e-06,
  "WP1": 7.346739531762215e-06,
  "WN2": 3.0587266286604007e-06,
  "WP2": 3.038069119888764e-06,
  "L1": 2.762855924312858e-08,
  "L2": 1.228588215527502e-07,
  "VDD": 0.8906585278880905,
  "Temp": 40.103664373134855,
  "V_vdd": 0.8906628,
  "V_gnd": 0.0,
  "V_in": 0.8906585,
  "V_out": 1.205001,
  "V_target": 0.9205885,
  "I_vdd": 0.00193829,
  "I_gnd": -0.0032357769343999998,
  "I_in": 0.0009869179,
  "I_out": 0.0003105529,
  "I_target": 1.61344e-08
},
```

Figure 3: Dataset in JSON Format

2.2 Graph Neural Networks

Graph Neural Networks are a type of neural network that uses a complete graph structure, rather than a single data sample, as one input. Therefore, a Graphical Neural Network is a logical choice for current prediction because a circuit is basically already a graph. Figure 4 is a graphical representation of the Cascaded Inverter Chain circuit previously shown in Figure 2.

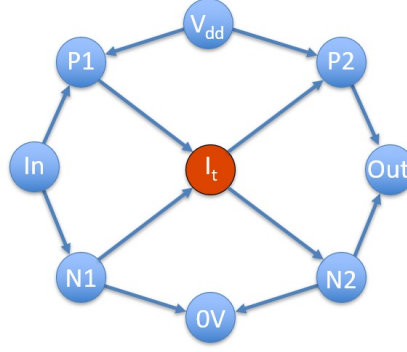


Figure 4: Graphical Representation of the Cascaded Inverter Chain

Mathematically, the graph is represented as a weighted adjacency matrix and a feature matrix whose rows are the feature vectors of each individual node in the graph. Learning can then be done to do classification or regression on node labels, edge weights, or an aggregate graph level. [11] Creating a graphical model of a circuit can be approached in several different ways. In physics problems, a circuit is typically translated to graph form by treating partitions of wire with equal potential as the nodes. For this case, however, it is preferable to encode circuit components as nodes to limit feature data to nodes and label data to edges, respectively. The feature matrix is shown in Table 1. (Note that the P1 and N1 nodes shown in Figure 4 are simplified into M1, and the P2 and N2 nodes are simplified into M2.) This allows the model to learn the relationship between component features and current flow, rather than relying on the model to infer the presence of components from patterns in the adjacency matrix. The adjacency matrix is shown in Table 2. It also allows for more flexible modeling of complex circuit topologies, where multiple components may be connected in parallel or series, without needing to explicitly encode these configurations in the graph structure.

	WP	WN	L	VDD	VTH0_P	...	VTH0_N	...	Temp	SkewL	SkewR	Size	Option
Vi	0	0	0	0	0	...	0	...	27	1	1	45	2
M1	2.0	1.0	0.045	0	-0.30	...	0.28	...	27	1	1	45	2
VDD	0	0	0	1.0	0	...	0	...	27	1	1	45	2
M2	4.0	2.0	0.045	0	-0.30	...	0.28	...	27	1	1	45	2
Vo	0	0	0	0	0	...	0	...	27	1	1	45	2
GND	0	0	0	0	0	...	0	...	27	1	1	45	2

Table 1: Feature Matrix

	Vi	M1	VDD	M2	Vo	GND
Vi	0	1.2e-7	0	0	0	0
M1	0	0	0	7.9e-6	0	8.1e-6
Vdd	0	8.1e-6	0	8.1e-6	0	0
M2	0	0	0	0	4.5e-6	8.1e-6
Vo	0	0	0	0	0	0
GND	0	0	0	0	0	0

Table 2: Adjacency Matrix

Graph Neural Networks may consist of many different types of layers, but the one that is most central to the concept is known generally as the message passing layer. This layer updates the features at each node, using information pulled from neighboring node features. For a general GNN this is not strictly

defined, although the edges typically encode how the “messages” are passed between nodes. While there are many types of GNNs, Graph Attention Networks (GATs) extend the basic message-passing framework by learning a scalar attention weight for each directed edge, so the model can adaptively weight the influence of each neighbor rather than treating all connections equally. [8] GATv2 corrects a rank-limitation in the original GAT by making the attention function strictly more expressive. [2] In GATv2, the attention score for a given edge (i, j) is computed as:

$$e_{ij} = \mathbf{a}^\top \text{LeakyReLU}(\mathbf{W}[\mathbf{h}_i \parallel \mathbf{h}_j]) \quad (1)$$

where:

- $\mathbf{h}_i$ and $\mathbf{h}_j$ are the feature vectors of the source and target nodes, respectively
- $[\mathbf{h}_i \parallel \mathbf{h}_j]$ denotes concatenation of the two feature vectors
- $\mathbf{W}$ is a learnable weight matrix applied to the concatenated features
- LeakyReLU is a non-linear activation function
- $\mathbf{a}$ is a learnable vector that projects the result to a scalar attention score

This formulation allows the attention mechanism to depend jointly on both nodes, making it more expressive than the original GAT, which used separate projections and summed them before the nonlinearity. In a circuit graph, nodes represent transistors and interconnects whose relative contributions to downstream current vary depending on operating point; learned per-edge attention weights are therefore a natural fit compared to the uniform aggregation of a plain GCN [5].

The training algorithm was implemented in Python using PyTorch and the PyG library for GATv2Conv layers. [1] To predict current values, regression on the graph’s edge weights with a GATv2 network are used. [2] For this problem, the graphs have been designed such that our training is entirely supervised. The model consists of three stages of GATv2Conv layers:

1. An input layer mapping node features to a 64-dimensional hidden space, using four attention heads whose outputs are averaged.
2. Three hidden layers of the same width and head configuration, each followed by an ELU activation.
3. An output layer with four heads whose outputs are *concatenated*, producing a 64-dimensional node embedding (16 dimensions per head).

The following sections explain the architecture each of these layers, the feature engineering and loss function, batching, the decoder, and optimization of the network. These are followed by the Experimentation section, which presents the results.

2.2.1 Input Layer

The input layer takes the raw node features, which may vary in dimension depending on the dataset, and projects them into a fixed 64-dimensional hidden space. This is accomplished using a GATv2Conv layer with four attention heads. The choice of four attention heads (which must be a power of two) is a balance between model expressiveness and computational efficiency. Using multiple heads allows the model to attend to different aspects of the neighborhood structure simultaneously, capturing diverse patterns in the data. Empirically, four heads provided sufficient capacity for the circuit prediction task without incurring excessive computational cost or overfitting. Fewer heads (e.g., two) may limit the model’s ability to capture complex relationships, while more heads (e.g., eight or more) increase memory and computation without a clear gain in performance for this dataset.

Each head computes its own attention-weighted aggregation of neighboring node features, and the outputs of the four heads are averaged. In the GATv2Conv implementation, each attention head produces an output vector of size 64. When using four heads with averaging, the outputs from

all heads are summed and divided by four, resulting in a single 64-dimensional vector per node. This is in contrast to concatenation, which would produce a 256-dimensional output (four heads $\times$ 64 dimensions each). Averaging preserves the original hidden size, ensuring the output remains 64-dimensional, providing a consistent embedding size for all nodes regardless of the original feature dimension.

The hidden size of 64 is a common choice in graph neural networks, providing enough capacity to encode rich node representations while keeping the model tractable for large graphs. It is large enough to capture the relevant features and interactions in the circuit graphs, but not so large as to risk overfitting or slow down training significantly. This value was selected based on standard practice and preliminary experiments.

Consistent embedding size is crucial because subsequent layers in the network expect a fixed-size input for every node, regardless of the original feature vector's dimension. This uniformity allows the model to process graphs with nodes of varying feature sets (e.g., different circuit components) and ensures compatibility with batch processing and downstream layers. By projecting all node features into the same 64-dimensional space, the model can generalize across different node types and datasets, facilitating transfer learning and robust performance.

2.2.2 Hidden Layers

The next three layers are also GATv2Conv layers, each with four attention heads and a hidden size of 64. The choice of three hidden layers is a balance between model depth and the risk of overfitting or vanishing gradients. With each additional layer, the model can aggregate information from nodes that are further away in the graph, enabling it to capture more complex dependencies. The rationale having four attention heads is similar to the input layer: multiple heads allow the model to capture diverse patterns and relationships in the graph. Four heads provide a good trade-off between expressiveness and computational cost, as validated by experiments. Using the same number of heads across layers also simplifies the architecture and ensures consistent dimensionality.

After each layer, an Exponential Linear Unit (ELU) activation is applied to introduce nonlinearity. Nonlinearity is essential in neural networks to allow the model to learn complex, non-linear relationships in the data. Without nonlinear activations, the network would be limited to learning only linear transformations, regardless of its depth. The Exponential Linear Unit (ELU) is chosen because it helps mitigate the vanishing gradient problem, speeds up learning, and can produce negative outputs, which may be beneficial for representing certain circuit features. ELUs often lead to faster convergence and better generalization compared to ReLU or other activations in some graph neural network applications. The equation for the ELU function is:

$$\text{ELU}(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

which means that when x is greater than 0, the function returns x , but when it is less than zero, it returns $\alpha(e^x - 1)$, where α is a hyperparameter that controls the saturation level for negative inputs. This hyperparameter is set to 1, which is typical for an ELU implementation. This ELU function produces to the graph shown in Figure 5.

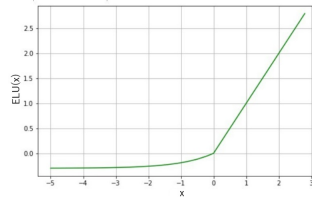


Figure 5: Exponential Linear Unit (ELU) Function [4]

The hidden layers allow the model to iteratively refine node representations by aggregating information from increasingly distant neighbors in the graph. The attention mechanism enables the model to focus on the most relevant nodes for each prediction, which is especially important in circuit graphs where certain nodes (such as supply rails or transistors) play more critical roles than others.

While graph size can influence architectural choices, the decisions here are primarily based on standard practice in graph neural networks. The circuit graphs in this work are of moderate size, so three layers are sufficient to capture relevant dependencies without excessive depth. The number of attention heads and the use of ELU are chosen for their general effectiveness rather than being tightly coupled to graph size. For much larger or more complex graphs, deeper networks or different configurations might be necessary.

2.2.3 Output Layer

The final GATv2Conv layer also uses four attention heads, but instead of averaging their outputs, it concatenates them. The rationale for using four attention heads in the output layer is consistent with the earlier layers: multiple heads allow the model to capture different aspects of the node’s neighborhood and learn a more nuanced representation. Four heads provide a good balance between model expressiveness and computational efficiency, as established in the input and hidden layers.

In the output layer, concatenation is used instead of averaging to preserve the unique information captured by each attention head. While averaging blends the outputs into a single vector (as in the input and hidden layers), concatenation retains the distinct features learned by each head, resulting in a richer and more expressive final embedding. This is particularly valuable for the final prediction, as it allows the decoder to leverage a broader set of features when estimating the target current. Concatenation increases the dimensionality (four heads $\times$ 16 dimensions = 64), but this is intentional to maximize the information available for the final regression task.

This results in a 64-dimensional node embedding, with each head contributing 16 dimensions. The concatenated output provides a richer and more expressive representation, which is then used by the decoder to predict the target edge current. This design allows the model to capture a diverse set of features from different attention heads, improving its ability to generalize across different circuit configurations.

2.2.4 Feature Engineering and Loss Function

Feature engineering encompasses all transformations applied to both input features and target labels to make the data suitable for model training. For the input features, two main strategies are used: 1) Convert descriptive parameters to numerical encodings, using their implied ordinality. For example, Skew is converted to a two-dimensional parameter describing the corner placement, and Option is converted to a one-dimensional parameter encoding low, medium, or high power. The PVT parameters are dropped due to redundancy with other numerical parameters. 2) To address imbalance in feature scale, features are standardized across the entire dataset to a mean of zero and variance of one. This prevents cases of exploding gradient during training.

For the target labels (currents), label normalization is performed as a specialized form of feature engineering. The five branch currents in the dataset span approximately six orders of magnitude (8 pA to 46 μ A). Training directly on raw current values with MAPE or MAE loss produced unstable gradients and poor convergence on small-magnitude currents. To address this, all current labels are normalized using a two-step process:

$$\hat{y} = \frac{\log_{10}(|I|) - \mu}{\sigma} \quad (2)$$

where μ and σ are computed from the flattened log-magnitudes of all current values across the full training set. This compresses the six-order dynamic range to a near-unit-scale output target. At evaluation time the prediction is inverse-transformed as $I = 10^{\hat{y}\sigma + \mu}$. With log-normalized labels, standard L1 loss (mean absolute error) becomes equivalent to a log-space relative error:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i| \quad (3)$$

Critically, the loss is computed only on I_{target} edges. At inference time the model receives only transistor dimensions and process/PVT parameters. That is, no other branch currents are available because those could only be obtained through simulation results. Masking the loss to I_{target} ensures the gradient signal reflects the *actual* prediction objective and avoids dilution from the five other edge types (I_{vdd} , I_{gnd} , I_{in} , I_{out} , and their junction duplicates).

2.2.5 Batching

In standard machine learning, all data samples (e.g., images) have the same shape. However, in graph learning, each graph can have a different number of nodes and edges, so their feature and adjacency matrices have different sizes. Because of this inconsistency, it's not possible to use standard batching techniques. The solution is to merge all of the graphs in a batch into a single, larger "super-graph" by concatenating all of the node feature matrices vertically, then placing each graph's adjacency matrix along the diagonal of a large, block-diagonal matrix (with zeroes elsewhere). This way, the batch is represented as one large graph, and the GAN can process all graphs in the batch together efficiently. At low batch sizes typical problems with stability were observed and large batch sizes led to a notable increase in measured error, so a batch size of 32 was chosen as a balance between stability and training speed.

2.2.6 Decoder

After the GATv2 encoder processes the graph, each node is represented by a 64-dimensional embedding vector $\mathbf{z}_i$ that encodes its physical, electrical, and topological context. The decoder's job is to predict the current for each edge (u, v) using these node embeddings. To do this, the decoder constructs a feature vector for each edge by combining information from both endpoint nodes in several ways:

- $\mathbf{z}_u$: Embedding of the source node (e.g., a transistor or interconnect)
- $\mathbf{z}_v$: Embedding of the target node
- $\mathbf{z}_u - \mathbf{z}_v$: Element-wise difference, capturing the relative relationship between the nodes
- $\mathbf{z}_u \odot \mathbf{z}_v$: Element-wise (Hadamard) product, capturing interactions between features

These four 64-dimensional vectors are concatenated to form a single 256-dimensional input vector for each edge:

$$[\mathbf{z}_u \parallel \mathbf{z}_v \parallel \mathbf{z}_u - \mathbf{z}_v \parallel \mathbf{z}_u \odot \mathbf{z}_v] \in \mathbb{R}^{256} \quad (4)$$

This design allows the decoder to leverage both the individual properties of each node and their pairwise relationships, which is important for accurately modeling current flow that depends on both local and contextual circuit features. The 256-dimensional edge feature vector is then passed through a two-layer multilayer perceptron (MLP):

- **First layer:** Fully connected, $256 \rightarrow 64$, followed by a ReLU activation. This layer learns nonlinear combinations of the edge features.
- **Second layer:** Fully connected, $64 \rightarrow 1$, producing a single scalar output $\hat{y}_{uv}$, which is the predicted log-normalized current for that edge.

Mathematically, the decoder can be written as:

$$\hat{y}_{uv} = \text{MLP}([\mathbf{z}_u \parallel \mathbf{z}_v \parallel \mathbf{z}_u - \mathbf{z}_v \parallel \mathbf{z}_u \odot \mathbf{z}_v]) \quad (5)$$

This approach is well-suited for edge regression tasks in graph neural networks, as it allows the model to flexibly capture both additive and multiplicative interactions between node features, as well as their

differences, which can be critical for predicting physical quantities like current that depend on the relationship between two circuit elements. The use of a two-layer MLP provides sufficient capacity to model complex nonlinear dependencies while remaining computationally efficient.

2.2.7 Optimization

The model is trained with the Adam optimizer at an initial learning rate of 1×10^{-4} . The scheduler, which is based on `ReduceLROnPlateau`, halves the learning rate after eight consecutive epochs without improvement in test loss, with a minimum learning rate of 1×10^{-6} . Training runs for 100 epochs with a batch size of 32 graphs, on a 70/30 train/test split of the dataset (70,000 training, 30,000 test samples). Training is performed on the OSU College of Engineering’s HPC cluster.

3 Related Work

Our primary reference sources for our privacy strategies were: *GAP: Differentially Private Graph Neural Networks with Aggregation Perturbation* by Sina Sajadmanesh et al and *Toward Representing Identical Privacy-Preserving Graph Neural Network via Split Learning* by Yiming Fang et al. [7], [3]

4 Approach

Our approach was to implement both Differential Privacy and Split Learning, separately and together, then attack them with three different attack methods. These strategies are not mutually exclusive, so our goal was to see whether they complement each other and provide additional privacy.

4.1 Differential Privacy

Differential Privacy (DP) is a training-time defense that limits how much influence any single sample can have on the learned model, which reduces the risk that an attacker can infer whether a specific record was in the training set. In this project, DP was implemented using DP-SGD style updates: per-step gradients are clipped to a fixed norm bound and then Gaussian noise is added before the optimizer step, so updates are intentionally “blurred” while still preserving useful learning signal.

Practically, this means the model is optimized with controlled sensitivity rather than raw gradients, creating a privacy-utility trade-off where stronger clipping/noise improves privacy but can degrade final predictive accuracy. The DP mode in our experiments used this noisy, clipped training path as a direct counterpart to baseline (non-private), split-learning, and combined modes, enabling apples-to-apples comparison of privacy attack outcomes, runtime overhead, and prediction quality. As expected, DP mainly reduced attack effectiveness by making model outputs less tied to individual samples, while introducing some utility cost visible in downstream error metrics.

Our hypothesis, based on the reference paper, was that DP would have a 20-30% impact on accuracy and no impact on latency.

4.2 Split Learning

Split Learning (SL) is a collaborative training approach that improves privacy by dividing the model across trust boundaries so no single party sees the full input-processing pipeline. In this project, the implementation separates foundry-specific process information from design-house training: a foundry-side encoder transforms sensitive process/model parameters into compact embeddings, and only those embeddings are shared with the design-side network that performs current prediction.

This means raw foundry parameters are not directly exposed to the main training loop or downstream artifacts, while the design model still receives enough signal to learn useful behavior. Operationally, the SL path uses dedicated split-training and split-inference code that saves design-house and foundry checkpoints separately and then evaluates the same privacy attacks and utility metrics as baseline/DP for consistent comparison. The intended effect is to reduce direct leakage of proprietary process details by constraining what intermediate information crosses parties, with a corresponding trade-off in pipeline complexity and potential runtime overhead.

Our hypothesis, based on the reference paper, was that SL would have a 15-25% impact on latency and no impact on accuracy.

4.3 Both

Since DP and SL are not mutually exclusive, we decided to also implement both at the same time. Our hypothesis was that the accuracy impact would be similar to DP and the latency impact would be similar to SL.

4.4 Attack Methods

The three attacks we implement were Embedding Inversion (EI), Edge Reconstruction (ER), and Membership Inference (MI).

EI trains an attacker-side decoder from model outputs to hidden input features. Its metric is test MSE. A larger MSE means the attacker reconstructs features less accurately, which indicates better privacy.

ER trains an attacker to reconstruct edge-state labels (derived from I_* current columns using train-set quantile thresholds) from model outputs. Its metric is test macro-accuracy. A lower accuracy means reconstruction is harder, which indicates better privacy.

MI predicts whether a sample was in the training set from per-sample residual features. Its metric is AUC. Privacy is better when AUC is closer to chance (0.5), so leakage can be tracked by $|AUC - 0.5|$.

5 Experimental Results

All of the training runs were done using the same dataset (described in Section 2.1 above) and the same Python code base. The code repository can be found here: <https://github.com/YosemiteSamurai/CurrentPrediction>. All experiments were tracked and visualized using Weights & Biases: <https://wandb.ai/yosemitesamurai/CurrentPrediction>. The $y - axis$ on the plots was cropped to highlight the difference.

The following sections describe the changes in model performance, the results of the privacy attacks, and the impact to training latency.

5.1 Model Performance

The training loss is shown in Figure 6 and the mean relative error is shown in Figure 7.



Figure 6: Training Loss

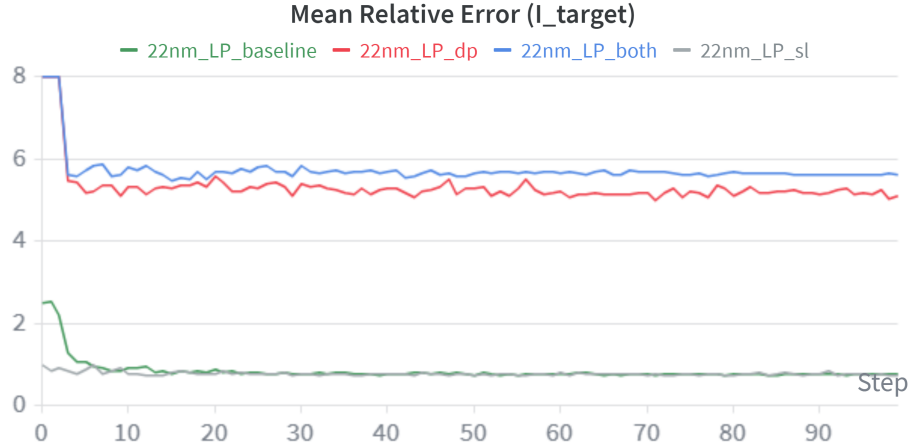


Figure 7: Mean Relative Error

Since MRE is the primary evaluation metric, the value for each at epoch 100 is shown in Table 3. A comparison to the baseline model is shown in Table 4.

Metric	Baseline	Split Learning	Differential Privacy	Both
MRE @ epoch 100	0.7585	0.7504	5.0917	5.6268

Table 3: Mean Relative Error at Epoch 100 for 22nm_LP

Mode	Absolute Δ MRE	Percentage Δ vs Baseline
Split Learning	-0.0081	-1.06%
Differential Privacy	+4.3332	+571.27%
Both	+4.8682	+641.81%

Table 4: MRE Change vs Baseline at Epoch 100 (22nm_LP)

As expected, the two versions that include Differential Privacy performed worse than the baseline, and the version that used only Split Learning performed similarly to the baseline.

5.1.1 DP-SGD Privacy Budget Sweep

We also ran a sweep to see how the privacy budget epsilon affects accuracy. We kept the model, the data split, and the optimizer the same, and only changed epsilon. We used a 10,000-sample subset of the 22nm_LP dataset with a 70/30 train/test split, trained for 100 epochs, and tested $\epsilon \in \{0.5, 1, 4, 16, 64, 256\}$. A smaller epsilon means more noise and stronger privacy. We repeated the sweep for two gradient clipping norms, $C = 1$ and $C = 4$. The noise level σ for each epsilon was set using the Opacus privacy accountant.

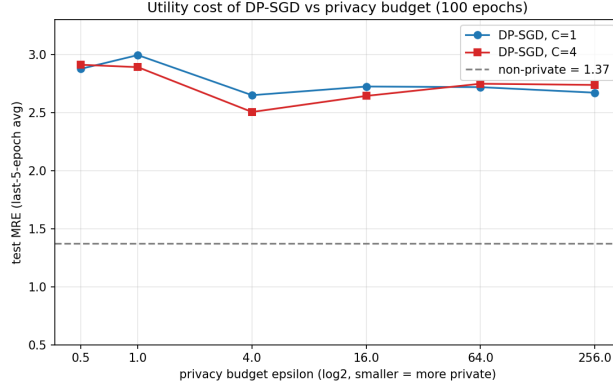


Figure 8: Test MRE versus privacy budget ϵ for DP-SGD at clipping norms $C = 1$ and $C = 4$.

Figure 8 shows two things. First, DP roughly doubles the mean relative error (MRE), from about 1.37 for the baseline to about 2.5 to 3.0 with DP. Second, this cost stays almost flat across the whole range of epsilon. Even at $\epsilon = 256$, where the noise is almost zero, the MRE is still about 2.7 and does not return to the baseline. Using a larger clipping norm ($C = 4$) does not help either. This tells us the accuracy cost comes mainly from the gradient clipping step in DP-SGD, not from the privacy noise. Table 5 lists the exact numbers for $C = 1$, and the MRE values here are a last-5-epoch average from this sweep.

Run	ϵ	Noise mult. σ	Last-5 MRE	Best MRE
Baseline	n/a	n/a	1.359	0.764
DP	0.5	5.31	2.875	2.211
DP	1	2.85	2.994	2.205
DP	4	1.03	2.649	1.746
DP	16	0.60	2.723	1.788
DP	64	0.42	2.718	1.711
DP	256	0.30	2.671	1.565

Table 5: DP-SGD privacy budget sweep at clipping norm $C = 1$ (100 epochs, 10,000-sample subset of 22nm_LP). MRE is the test mean relative error on I_{target} .

Figure 9 shows the training loss for the same runs. All the DP runs settle at a similar level above the baseline, and epsilon mainly affects how fast they get there.

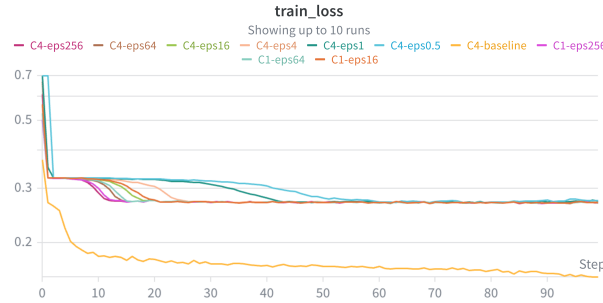


Figure 9: Per-epoch training loss for the DP-SGD runs.

5.2 Attack Results

The results are shown in Table 6. A comparison of each method’s model to the baseline model is shown in Table 7.

Relative to baseline, ER shows the clearest privacy improvement: Both > Differential Privacy >> Split Learning. MI remains close to chance for all modes, with small absolute differences and some instability between global vs holdout AUC. EI is mixed: Split Learning improves privacy by this metric, while Differential Privacy and both reduce EI MSE (easier inversion under this proxy), so EI should be treated as a secondary signal here.

Because raw metric scales differ, emphasize baseline-relative change. For this run, the strongest and most consistent privacy evidence is ER leakage reduction, especially for Both and Differential Privacy. Present MI as near-chance leakage and EI as mixed, then pair these with timing tables and W&B accuracy charts to show the privacy-utility trade-off. The failure of MI to show any meaningful leakage was not surprising given the presentation of our colleagues that concluded that MI isn’t generally an effective attack.

Metric	Baseline	Split Learning	Diff. Privacy	Both
EI MSE	0.6618	0.6947	0.5658	0.5513
EI 95% CI	[0.6565, 0.6667]	[0.6894, 0.7006]	[0.5605, 0.5705]	[0.5470, 0.5564]
ER accuracy	0.6513	0.6425	0.5652	0.5281
ER 95% CI	[0.6487, 0.6535]	[0.6405, 0.6445]	[0.5628, 0.5676]	[0.5257, 0.5304]
MI AUC (global)	0.5035	0.5042	0.5033	0.5027
MI AUC (holdout)	0.5032	0.4977	0.5041	0.4946

Table 6: Privacy Attack Metrics for 22nm_LP (raw values)

Metric (positive = more private)	Split Learning	Diff. Privacy	Both
EI privacy change (%)	+4.97	-14.51	-16.70
ER privacy gain (% leakage reduction)	+1.35	+13.22	+18.92
MI privacy gain (% leakage-gap reduction)	-20.00	+5.71	+22.86

Table 7: Privacy Change vs Baseline for 22nm_LP

Notes on Table 7: EI privacy change (%) is $(\text{MSE}_{\text{mode}} - \text{MSE}_{\text{base}}) / \text{MSE}_{\text{base}}$, ER privacy gain (% leakage reduction) is $(\text{Acc}_{\text{base}} - \text{Acc}_{\text{mode}}) / \text{Acc}_{\text{base}}$, and MI privacy gain (% leakage-gap reduction) is $(|\text{AUC}_{\text{base}} - 0.5| - |\text{AUC}_{\text{mode}} - 0.5|) / |\text{AUC}_{\text{base}} - 0.5|$.

5.3 Training Latency

The training latency statistics are shown below in Table 8, Table 9, and Table 10.

As expected, the Data Prep / Load time was significantly higher for the two runs that included Split Learning. Surprisingly, though, these runs had shorter training time that more than compensated for it. However, it’s not reasonable to draw the conclusion that Split Learning leads to faster training times since only one run was done for each combination. Proving that would require enough runs to constitute a statistically significant sample.

A conclusion we *can* draw from the data is that the Data Prep / Load times are much smaller than the Training times, so using Split Learning probably plays less of a factor in overall runtime than congestion on the HPC cluster on which they’re being trained.

Metric	Baseline	Split Learning	Diff. Privacy	Both
Data Prep / Load (s)	0.07	2.76	0.07	2.53
Training (s)	4842	4095	4789	4744

Table 8: Model Run Timing Comparison for 22nm_LP

Metric	Split Learning	Diff. Privacy	Both
Data Prep / Load Change vs Baseline (%)	+3842.86	+0.00	+3514.29
Training Change vs Baseline (%)	-15.43	-1.10	-2.03
Avg Epoch Change vs Baseline (%)	-15.49	-1.09	-2.07
Training Speedup vs Baseline (x)	1.18	1.01	1.02
Avg Epoch Speedup vs Baseline (x)	1.18	1.01	1.02

Table 9: Derived Timing Comparison vs Baseline (22nm_LP)

Metric	Split Learning	Diff. Privacy	Both
Data Prep / Load Δ (s)	+2.69	+0.00	+2.46
Training Δ (s)	-747.45	-53.08	-98.18
Avg Epoch Δ (s)	-7.50	-0.53	-1.00

Table 10: Absolute Timing Deltas vs Baseline (22nm_LP)

6 Discussion & Conclusions

We believe our approach allowed us to effectively compare the effectiveness of Split Learning and Differential Privacy in this context. However, it would be unwise to form general conclusions about these methods based solely on our results. Our data show that the attack results are not significantly worse when employing these methods, individually or together, compared to the baseline. More robust attacks may tell a different story.

Furthermore, future versions of the Current Prediction Model that include multiple designs (see Section 2.1) might be more susceptible to attack and get more protection from these privacy methods. This may also be true of the latency. Training a model with a much larger dataset (due to it including more designs) might reveal the additional latency to be significant.

Quantitatively, our hypotheses in Section 4 were way off. The impact DP had on accuracy was on the order of 600% (not 20-30%). The impact that SL had on latency was effectively zero (not 15-20%) because the impact was all in the data prep / load phase, which was not a significant contributor to overall latency. Qualitatively, we expected DP to impact accuracy and not latency and for both to carry over the impact of DP on accuracy and SL on latency. This proved to be correct.

We believe that our work demonstrates the following key learnings:

1. Differential Privacy and Split Learning have fundamentally different impacts on models and they do not necessarily complement each other. As shown in Table 7, Both performs worse than each perform individually for Embedding Inversion. However, under the Edge Reconstruction attack, Both performs better than each does individually.
2. There is no “one best” privacy method nor “one best” attack method. How these defenses and attacks perform is going to be a function of the model, so it is up to the model owner to accordingly. That is, the model owner must understand the threats their model is likely to face and put up the best defense.
3. Defensive strategies have clear trade-offs. It is not wise to simply employ every defensive strategy possible since they do not necessarily complement each other and could have zero return on investment. The investment, of course, is training time and accuracy..

Contributions

Since this project was based on Matthew Jones's PhD research and the other two team members are undergraduate students, some amount of disproportionality was to be expected. Aaryan and Ren contributed by researching and implementing (in Python) Split Learning and Differential Privacy, respectively, then Matt integrated their code into his, trained the model, produced the results, and wrote the report (except for Section 5.1.1). Matt also prepared the slides for all three team presentations.

References

- [1] Pyg documentation, 2026. [Online; accessed March 18, 2026].
- [2] S. Brody, U. Alon, and E. Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022.
- [3] Y. Fang, H. Jiao, and R. Huang. Toward representing identical privacy-preserving graph neural network via split learning. *IEEE Access*, 2024.
- [4] GeeksforGeeks. Elu activation function in neural network. <https://www.geeksforgeeks.org/deep-learning/elu-activation-function-in-neural-network/>, July 2025. Accessed: 2026-05-20.
- [5] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *CoRR*, abs/1609.02907, 2016.
- [6] Microelectronics Research Center, University of Minnesota. Predictive technology model (ptm). <https://mec.umn.edu/ptm/#h.v5g6eyuivo9x>, n.d. Accessed: 2026-03-16.
- [7] S. Sajadmanesh, A. S. Shamsabadi, A. Bellet, and D. Gatica-Perez. Gap: Differentially private graph neural networks with aggregation perturbation. *arXiv preprint arXiv:2203.00949*, 2022.
- [8] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [9] Wikipedia contributors. Ngspice — wikipedia, the free encyclopedia. <https://en.wikipedia.org/wiki/Ngspice>, 2026. Accessed: 2026-05-20.
- [10] Wikipedia contributors. Spice — wikipedia, the free encyclopedia. <https://en.wikipedia.org/wiki/SPICE>, 2026. Accessed: 2026-05-20.
- [11] J. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, and M. Sun. Graph neural networks: A review of methods and applications. *CoRR*, abs/1812.08434, 2018.

Appendix

SPICE Netlists

The SPICE netlists used for the simulations are below. “2inv.sp” is the netlist for the circuit shown in Figure 2. “template.sp” is used to parameterize the transistor sizes, power supply voltage, and operating temperature.

2inv.sp

```
* =====
* Cascaded Inverter Testbench for ML Dataset
* All node currents and voltages measurable
* =====

.option post
.option nomod
.option accurate
.option method=gear

* ----- Subcircuits for current measurement -----

.subckt PMOS_MEAS D G S B Dmeas 0 PARAMS: WP=1u L=100n
Vmeas_p D Dmeas 0
M1 Dmeas G S B pmos W={WP} L={L}
.ends PMOS_MEAS

.subckt NMOS_MEAS D G S B Dmeas 0 PARAMS: WN=1u L=100n
Vmeas_n D Dmeas 0
M1 Dmeas G S B nmos W={WN} L={L}
.ends NMOS_MEAS

.subckt C_MEAS N1 N2 N1meas 0 PARAMS: CVAL=1f
Vmeas_c N1 N1meas 0
C1 N1meas N2 {CVAL}
.ends C_MEAS

* ----- Driver Inverter -----
XMP1 target_int in vdd vdd target_mp1 0 PMOS_MEAS PARAMS: WP={WP1} L={L1}
XMN1 target_int in 0 0 target_mn1 0 NMOS_MEAS PARAMS: WN={WN1} L={L1}
Vmeas_target target target_int 0V

* ----- Load Inverter -----
XMP2 out_meas target vdd vdd out_mp2 0 PMOS_MEAS PARAMS: WP={WP2} L={L2}
XMN2 out_meas target 0 0 out_mn2 0 NMOS_MEAS PARAMS: WN={WN2} L={L2}
Vmeas_out out out_meas 0

* ----- Load Cap (optional realism) -----
XCload out 0 out_cload 0 C_MEAS PARAMS: CVAL=5f

* ----- Analysis -----
.tran 1p 5n

* ----- Model Files -----
.include models/22nm_HP.pm

* ----- Output -----
.control
run
print v(vdd) v(in) v(out) v(target)
print vmeas_vdd#branch vmeas_in#branch vmeas_out#branch vmeas_target#branch
print tran I(vmeas_vdd) I(vmeas_in) I(vmeas_out) I(vmeas_target)
.endc

* ----- Supplies -----
VDD vdd_src 0 {VDD}
Rsupply vdd_meas vdd_src 0.01
Vmeas_vdd vdd vdd_meas 0

* ----- Input -----
Vin in_src 0 PULSE(0 {VDD} 0 20p 20p in 2n)
Rin in_meas in_src 0.01
Vmeas_in in in_meas 0

* ----- Ground connection (forced to 0)
* gnd is now directly connected to SPICE ground
* Remove Rgnd, connect gnd to 0 with a wire
* In SPICE, just use the same node name for both:
```

* All references to 'gnd' are now node 0

template.sp

* Auto-generated

```
.param WN1={WN1}
.param WP1={WP1}
.param WN2={WN2}
.param WP2={WP2}
.param L1={L1}
.param L2={L2}

.param VDD={VDD}
.param TEMP={TEMP}

.include "{model}"
.include "{pvt_corner}"
.include "{skew_corner}"
```

* All measurements are defined in 2inv.sp
.include "2inv.sp"

PTM Model Example

An example PTM model file is below.

22nm_LP.pm

* PTM Low Power 22nm Metal Gate / High-K / Strained-Si
* nominal Vdd = 0.95V

.model nmos nmos level = 54

+version = 4.0	binunit = 1	paramchk= 1	mobmod = 0
+capmod = 2	igcmmod = 1	igbmod = 1	geomod = 1
+diommod = 1	rdsmod = 0	rbodymod= 1	rgatemod= 1
+permod = 1	acnqsmode= 0	trnqsmode= 0	
+tnom = 27	tox = 1.4e-009	tox = 1.1e-009	tox = 1.4e-009
+dtom = 3e-010	epsrox = 3.9	wint = 5e-009	lint = 0
+ll = 0	wl = 0	lln = 1	wln = 1
+lw = 0	ww = 0	lwn = 1	wwn = 1
+lwl = 0	wwl = 0	xpart = 0	toxref = 1.4e-009
+vth0 = 0.68858	k1 = 0.4	k2 = 0	k3 = 0
+k3b = 0	w0 = 2.5e-006	dvt0 = 1	dvt1 = 2
+dvt2 = 0	dvt0w = 0	dvt1w = 0	dvt2w = 0
+dsb = 0.1	minv = 0.05	voff1 = 0	dvt0 = 1e-011
+dvt1 = 0.1	lpe0 = 0	lpeb = 0	xj = 7.2e-009
+ngate = 1e+023	ndep = 5.5e+018	nsd = 2e+020	phin = 0
+cdsc = 0	cdscb = 0	cdscd = 0	cit = 0
+voff = -0.1092	nfactor = 1.6	eta0 = 0.0105	etab = 0
+vfb = -0.55	u0 = 0.035	ua = 6e-010	ub = 1.2e-018
+uc = 0	vsat = 170000	a0 = 1	ags = 0
+a1 = 0	a2 = 1	b0 = 0	b1 = 0
+keta = 0.04	dwg = 0	dwb = 0	pclm = 0.02
+pdiblc1 = 0.001	pdiblc2 = 0.001	pdiblc3 = -0.005	drout = 0.5
+pvag = 1e-020	delta = 0.01	pscbe1 = 8.14e+008	pscbe2 = 1e-007
+fprout = 0.2	pdits = 0.01	pditsd = 0.23	pditsl = 2300000
+rsh = 5	rdsw = 180	rsd = 75	rdw = 75
+rdswmin = 0	rdwmin = 0	rsdmin = 0	prwg = 0
+prwb = 0	wr = 1	alpha0 = 0.074	alpha1 = 0.005
+beta0 = 30	agidl = 0.0002	bgidl = 2.1e+009	cgidl = 0.0002
+egidl = 0.8	aigbacc = 0.012	bigbacc = 0.0028	cigbacc = 0.002
+nigbacc = 1	aigbinv = 0.014	bigbinv = 0.004	cigbinv = 0.004
+eigbinv = 1.1	nigbinv = 3	aigc = 0.015211	bigc = 0.0027432
+cigc = 0.002	aigsd = 0.015211	bigsd = 0.0027432	cigsd = 0.002
+nigc = 1	poxedge = 1	bigcd = 1	ntox = 1
+xrcrg1 = 12	xrcrg2 = 5		
+cgso = 6.5e-011	cgdo = 6.5e-011	cgbo = 2.56e-011	cgdl = 2.653e-010
+cgsl = 2.653e-010	ckappas = 0.03	ckappad = 0.03	acde = 1
+moin = 15	noff = 0.9	voffcv = 0.02	

```

+kt1    = -0.11          kt1l    = 0          kt2    = 0.022          ute    = -1.5
+ua1    = 4.31e-009      ub1    = 7.61e-018      uc1    = -5.6e-011     prt    = 0
+at      = 33000

+fnoimod = 1            tnoimod = 0

+jss     = 0.0001        jsws   = 1e-011        jswgs  = 1e-010        njs    = 1
+ijthsfwd= 0.01         ijthsrev= 0.001        bvs    = 10           xjbvs  = 1
+jsd     = 0.0001        jswd   = 1e-011        jswgd  = 1e-010        njd    = 1
+ijthdfwd= 0.01         ijthdrev= 0.001        bvd    = 10           xjbvd  = 1
+pbs     = 1             cjs    = 0.0005        mjs    = 0.5          pbsws  = 1
+cjsws   = 5e-010        mjsws  = 0.33         pbswgs = 1           cjswgs  = 3e-010
+mjswgs  = 0.33         pbd    = 1           cjd    = 0.0005        mjd    = 0.5
+pbswd   = 1            cjswd   = 5e-010        mjswd  = 0.33         pbswgd = 1
+cjswgd  = 5e-010        mjswgd = 0.33         tpb    = 0.005        tcj    = 0.001
+tpbsw   = 0.005        tcjsw  = 0.001        tpbswg = 0.005        tcjswg = 0.001
+xtis    = 3            xtids   = 3

+dmcg    = 0            dmci    = 0            dmdg   = 0            dmcgt  = 0
+dwj     = 0            xgw     = 0            xgl    = 0

+rrshg   = 0.4          gbmin   = 1e-010        rbpb   = 5            rbpd   = 15
+rbps    = 15          rbdb    = 15          rbsb   = 15          ngcon  = 1

.model pmos pmos level = 54

+version = 4.0          binunit = 1          paramchk= 1          mobmod = 0
+capmod  = 2            igcmmod = 1          igbmod  = 1          geomod = 1
+diomod  = 1            rdsmod  = 0          rbodmod = 1          rgatemod= 1
+permod  = 1            acnqsmod= 0          trnqsmod= 0

+tnom     = 27          tox     = 1.4e-009        toxp    = 1.08e-009    toxm    = 1.42e-009
+dttox   = 3.2e-010    epsrox  = 3.9          wint    = 5e-009        lint    = 0
+l1       = 0           wl       = 0           lln     = 1           wln     = 1
+l1w      = 0           ww       = 0           lwn     = 1           wwn     = 1
+l1wl     = 0           wwl      = 0           xpart   = 0          toxref  = 1.42e-009

+vth0     = -0.63745    k1      = 0.4          dvt0    = 1           dvt1    = 2
+k3b      = 0           w0      = 2.5e-006    dvt1w   = 0           dvt2w   = 0
+dvt2     = -0.032      dvt0w   = 0           voffl   = 0           dvtp0    = 1e-011
+dsb      = 0.1         minv    = 0.05        lpeb    = 0           xj       = 7.2e-009
+dvtp1    = 0.05        lpe0    = 0           nsd     = 2e+020        phin    = 0
+ngate    = 1e+023      ndep    = 4.4e+018    cdsd    = 0           cit      = 0
+cjsc     = 0           nfactor = 1.8         eta0    = 0.0105       etab    = 0
+voff     = -0.09       u0      = 0.011        ua      = 2e-009        ub       = 5e-019
+vfb      = 0.55       vsat    = 170000      a0      = 1           ags     = 1e-020
+uc       = 0           a2      = 1           b0      = 0           b1      = 0
+a1       = 0           dwg     = 0           dwb     = 0           pclm    = 0.12
+keta     = -0.047      pdiblc1 = 0.001        pdiblc2 = 0.001        drout   = 0.56
+pdiblc1  = 0.001      delta   = 0.01        pdits   = 0.08         psbce1  = 8.14e+008    psbce2  = 9.58e-007
+pvag     = 1e-020      pdits   = 0.08        rsw     = 72.5         pdits1  = 2300000
+fprount  = 0.2        rdsw    = 230         rdw     = 72.5         rdw     = 72.5
+rrsh     = 5          rdswmin = 0          rswmin  = 0           prwg    = 0
+rdswmin  = 0          wr       = 1          alpha0  = 0.074        alpha1  = 0.005
+prwb     = 0          agidl   = 0.0002      bgidl   = 2.1e+009      cgidl   = 0.0002
+beta0    = 30         aigbacc = 0.012        bigbacc = 0.0028       cigbacc = 0.002
+egidl    = 0.8         aigbinv = 0.014        bigbinv = 0.004        cigbinv = 0.004
+nigbacc  = 1          nigbinv = 3           aigc    = 0.0097       bigc    = 0.00125
+eigbinv  = 1.1        aigsd   = 0.0097      bigsd   = 0.00125      cigsd   = 0.0008
+cigc     = 0.0008      poxedg  = 1           pigcd   = 1           ntox    = 1
+nigc     = 1
+xrcrg1   = 12          xrcrg2  = 5

+cgso     = 6.5e-011    cgdo    = 6.5e-011    cgbo    = 2.56e-011    cgd1    = 2.653e-010
+cgs1     = 2.653e-010 ckappas = 0.03         ckappad = 0.03         acde    = 1
+moin     = 15          noff    = 0.9         voffcv  = 0.02

+kt1      = -0.11       kt1l    = 0          kt2      = 0.022       ute      = -1.5
+ua1      = 4.31e-009   ub1     = 7.61e-018   uc1      = -5.6e-011   prt      = 0
+at        = 33000

+fnoimod = 1            tnoimod = 0

+jss       = 0.0001      jsws    = 1e-011      jswgs   = 1e-010      njs      = 1
+ijthsfwd= 0.01         ijthsrev= 0.001      bvs      = 10          xjbvs    = 1
+jsd       = 0.0001      jswd    = 1e-011      jswgd   = 1e-010      njd      = 1
+ijthdfwd= 0.01         ijthdrev= 0.001      bvd      = 10          xjbvd    = 1
+pbs       = 1           cjs      = 0.0005     mjs      = 0.5         pbsws    = 1
+cjsws     = 5e-010      mjsws   = 0.33       pbswgs  = 1           cjswgs   = 3e-010
+mjswgs    = 0.33       pbd      = 1          cjd      = 0.0005     mjd      = 0.5

```

+pbswd = 1	cjswd = 5e-010	mjswd = 0.33	pbswd = 1
+cjswdgd = 5e-010	mjswdgd = 0.33	tpb = 0.005	tcj = 0.001
+tpbsw = 0.005	tcjsw = 0.001	tpbswg = 0.005	tcjswg = 0.001
+xtis = 3	xtid = 3		
+dmcg = 0	dmci = 0	dmdg = 0	dmcgt = 0
+dwj = 0	xgw = 0	xgl = 0	
+rshg = 0.4	gbmin = 1e-010	rbpb = 5	rbpd = 15
+rbps = 15	rbdb = 15	rbsb = 15	ngcon = 1

Circuit Description

The testbench is comprised of transistors, subcircuits, measurement sources, and supply and input sources. The transistors are what make up the inverters. The rest of the elements are to facilitate measurement for data capture. A description of each element is below.

Transistors

- *MP1* (PMOS, $W=WP1$, $L=L1$) is the pull-up transistor of the *driver inverter* (INV1). When the input is low, MP1 charges the target node toward VDD. Its width WP1 and length L1 are swept across the design space.
- *MN1* (NMOS, $W=WN1$, $L=L1$) is the pull-down transistor of the driver inverter. When the input is high, MN1 discharges target toward ground. Together with MP1, it sets the switching threshold and transition speed of INV1.
- *MP2* (PMOS, $W=WP2$, $L=L2$) is the pull-up transistor of the *load inverter* (INV2). Driven by the target node, it charges the out node. Its dimensions WP2 and L2 are independently swept.
- *MN2* (NMOS, $W=WN2$, $L=L2$) is the pull-down transistor of the load inverter. Together with MP2, it presents a realistic capacitive and resistive load to the driver inverter, which directly affects the peak current drawn from VDD.

Subcircuits

- *XMP1: PMOS_MEAS* (contains MP1 + V_{p1}) wraps the PMOS transistor with a 0 V measurement voltage source (V_{p1}) in series with its drain. This allows Ngspice simulations to report the instantaneous drain current of MP1 as a branch current without disturbing the circuit.
- *XMN1: NMOS_MEAS* (contains MN1 + V_{n1}) uses the same technique for the NMOS transistor of INV1, enabling measurement of MN1's drain current via V_{n1} .
- *XMP2: PMOS_MEAS* (contains MP2 + V_{p2}) is the drain current measurement wrapper for MP2 in the load inverter.
- *XMN2: NMOS_MEAS* (contains MN2 + V_{n2}) is the drain current measurement wrapper for MN2 in the load inverter.
- *XCload: C_MEAS* (contains C_{load} + $V_{m,c}$) wraps the 5 fF load capacitor with a 0 V measurement source. This captures the current flowing into the capacitive load, representing the dynamic charging/discharging current that a downstream stage would see.

Measurement Sources (0 V)

- $V_{m,vdd}$ (Vmeas_vdd) measures the total current drawn from the VDD supply rail. Its peak value (I_{vdd}) captures supply current behavior relevant to power integrity and IR-drop risk.
- $V_{m,in}$ (Vmeas_in) measures the current delivered by the input source. Captures gate-tunneling and capacitive charging currents at the input node (I_{in}).
- $V_{m,tgt}$ (Vmeas_target) measures the current flowing from the driver inverter's output node (target_int) to the load inverter's gate (target). Records I_{target} , the primary prediction target for the DL model, reflecting the inter-stage drive current.
- $V_{m,out}$ (Vmeas_out) measures the current at the output of the load inverter (I_{out}), capturing the current flowing into the load capacitor path.
- V_{p1} , V_{n1} , V_{p2} , V_{n2} is internal to PMOS_MEAS / NMOS_MEAS subcircuits. Each measures the drain current of its respective transistor, enabling per-device current attribution.

- $V_{m,c}$ is internal to C_MEAS. Measures the capacitor charging current, distinguishing it from resistive leakage paths.

Supply and Input Sources

- V_{DD} is the DC voltage source providing the supply voltage. The value is swept across the design space to capture voltage-dependent behavior.
- R_{supply} is a small series resistance (0.01Ω) in the VDD path, modeling interconnect resistance and aiding convergence.
- V_{in} is the pulsed voltage source (PULSE) generating the switching stimulus. Rise/fall times are 20 ps; pulse width is 1 ns with a 2 ns period.
- R_{in} is a small series resistance (0.01Ω) in the input path, matching the VDD path topology.

The goal of the simulation is to measure the max voltage and peak current at each node in circuit, so it can be added to the dataset. The nodes are described below.

- vdd is a regulated supply rail, downstream of $V_{m,vdd}$. Voltage V_vdd is recorded to verify supply integrity; current through $V_{m,vdd}$ gives the supply current target.
- in is the gate input to INV1, downstream of $V_{m,in}$. Driven by a PULSE source (V_{in}) through R_{in} . Voltage V_in and current I_in are recorded.
- $target_int$ is an internal output of INV1 (junction of MP1 and MN1 drain paths), before the measurement source $V_{m,tgt}$.
- $target$ is the gate input to INV2, downstream of $V_{m,tgt}$. Voltage V_target and current I_target are recorded. This is the inter-stage node whose transition drives the peak supply current.
- out_meas is an internal output of INV2 (junction of MP2 and MN2 drain paths), before $V_{m,out}$.
- out is the final output node, downstream of $V_{m,out}$. Voltage V_out and current I_out are recorded.
- GND (node 0) is the global ground reference. I_gnd is derived by Kirchhoff's current law as the negative sum of all other node currents.

Table 11 shows the simulation results that will be captured as features in the dataset.

Feature	Source	Description
V_vdd	$V_{m,vdd}$, node vdd	Peak voltage at VDD rail
V_in	$V_{m,in}$, node in	Peak voltage at input
V_out	$V_{m,out}$, node out	Peak voltage at output
V_gnd	node 0	Ground reference (always 0)
V_target	$V_{m,tgt}$, node target	Peak voltage at inter-stage node
I_vdd	$V_{m,vdd}$ branch	Peak supply current
I_in	$V_{m,in}$ branch	Peak input current
I_out	$V_{m,out}$ branch	Peak output current
I_gnd	Kirchhoff (derived)	Ground return current
I_target	$V_{m,tgt}$ branch	Peak inter-stage current (prediction target)

Table 11: Simulation Results as Dataset Features

Simulation Methodology

The dataset was generated by simulating, using Ngspice, the circuit testbench across different process models, process corners and skews, and transistor sizes. Each simulation represents a unique combination of process technology, operating conditions, device asymmetry, and circuit parameters.

These variations are described below.

Process Models (9 variations) [6]. The simulation covers nine different semiconductor process technologies:

- *22nm HP*: Low threshold voltage, thin oxide for high performance 22nm node
- *22nm LP*: Higher threshold voltage for reduced leakage and low power 22nm node
- *32nm HP*: High performance 32nm node
- *32nm LP*: Low power 32nm node
- *45nm HP*: High performance 45nm node
- *45nm LP*: Low power 45nm node
- *65nm Bulk*: Traditional CMOS bulk process (neither HP nor LP)
- *90nm Bulk*: Mature bulk technology (neither HP nor LP)
- *130nm Bulk*: Legacy bulk process (neither HP nor LP)

PVT Corners (5 variations). Process, Voltage, Temperature combinations for comprehensive operating conditions:

- *Typical*: Nominal operating conditions
- *Slow*: Conservative: Low voltage, high temperature
- *Fast*: Aggressive: High voltage, low temperature
- *Hot*: High temperature stress testing
- *Cold*: Low temperature operation

Process Skew Corners (5 variations). Device asymmetry variations capturing realistic NMOS/PMOS device mismatches:

- *TT*: Balanced device performance
- *FF*: Both devices fast (high performance)
- *SS*: Both devices slow (low leakage)
- *FS*: Asymmetric: affects rise vs fall times
- *SF*: Asymmetric: affects fall vs rise times

Monte Carlo Device Parameters. For each combination above, the following parameters are randomly sampled:

- *WN1, WN2*: NMOS transistor widths (process-specific ranges)
- *WP1, WP2*: PMOS transistor widths (process-specific ranges)
- *L1, L2*: Channel lengths, 20nm-600nm (node dependent)
- *VDD*: Supply voltage, 0.7V-2.0V (node dependent)
- *TEMP*: Operating temperature, -20°C to 125°C
- *Example 22nm ranges*: W: 0.2 μ m - 8 μ m, L: 20nm - 200nm

Presentation Slides

The following pages are the slides from the Final Presentation given on 3 June 2026. Unfortunately, the results shown in Section 5 were not available in time for the presentation.



AI 539: Trustworthy Machine Learning Final Project Presentation

Matthew Jones, Aaryan Raghuvanshi, & Ren Yanghui

3 June 2026

Dr. Sanghyun Hong

1

Project Overview



Current Prediction in Analog ICs using GANs

- The goal is to determine certain analog circuit current values without time-consuming and costly simulations
- There are IP concerns regarding semiconductor manufacturing process data, so we want to keep it private
- We are planning to implement two different methods:
 - Split Encoder Architecture
 - Federated Learning with Differential Privacy
- These methods can be implemented separately or together

2

2

TML Project Goals



Specific:

- Establish a method for ensuring privacy of manufacturing data without significantly impacting accuracy or latency

General:

- Understand how different methods can be tuned to make the trade-off between privacy and accuracy / latency
- Understand the spectrum of “privacy” in different contexts

3

3

DP-SGD Hypothesis



- Reference: *GAP: Differentially Private Graph Neural Networks with Aggregation Perturbation*
 - <https://arxiv.org/pdf/2203.00949>
- Expecting impact on accuracy to be 20-30%
 - Epsilon(ϵ): smaller $\epsilon \rightarrow$ more gradient noise $\rightarrow$ stronger privacy, larger accuracy loss
 - GAP uses $\epsilon \in [1, 8]$ for similar GNN tasks; But exact impact on current prediction TBD
- No expected impact to latency

4

Split Learning Hypothesis



- Reference: *Toward Representing Identical Privacy-Preserving Graph Neural Network via Split Learning*
 - <https://ieeexplore.ieee.org/document/10711188>
- Expecting impact on latency to be 15-25%
 - Communication bottleneck created by transferring embedding vectors and gradients
- Negligible impact on model accuracy
 - No noise being injected during the training process

5

Combined Hypothesis



- Impacts
 - *Both* methods will contribute to increase *privacy*
 - The DP-SDG method will mostly impact the *accuracy*
 - The Split Learning method will mostly impact the *latency*
- We can tune both methods to maximize privacy while minimizing the impact to accuracy and latency

6

Privacy Attacks



Embedding inversion:

- Reconstructs the original input features from the model's internal embeddings
- If it works well, it means the representation still leaks a lot of raw information

Edge reconstruction:

- Infers the hidden graph structure or edge-level relationships from the model outputs
- If it works well, it means recovering which current relationships model encoded

Membership inference:

- Tells whether a specific sample was part of the training set
- A strong result means the model is memorizing enough to reveal membership

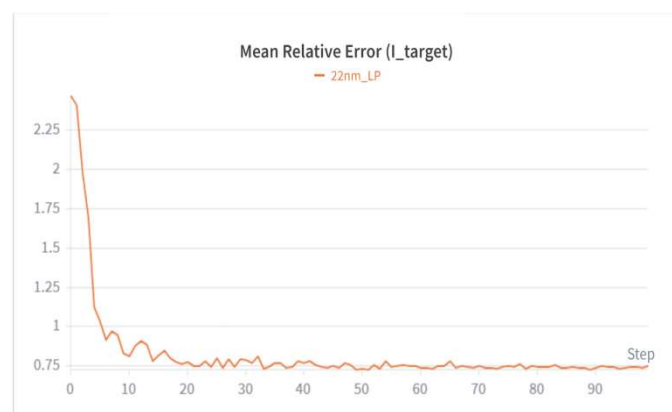
7

7

Baseline Results



- Our baseline for accuracy comparisons is the MRE for the 22nm_LP process
- It took ~2 hours to generate the dataset
- It took ~2 hours to train the model



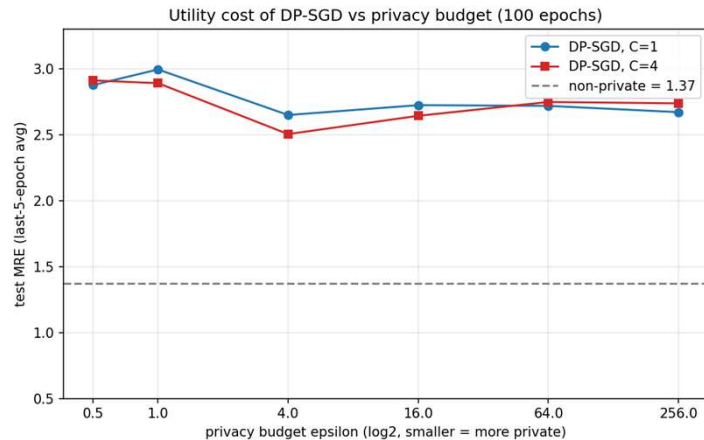
8

8

DP-SDG Test Results



- Varied ϵ from 0.5 to 256 using two clip norms ($C=1,4$)
- DP doubles MRE (~ 1.37 to ~ 2.7)
- Basically flat across all ϵ and clip norms



9

9

DP-SDG Test Results



- The new train_loss plot shows that all DP runs converge to the same floor
- Changes to ϵ only impacts how fast they converge

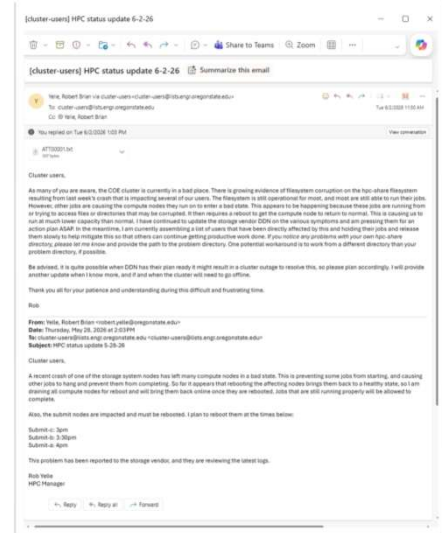


10

10

Results

- My quota on /nfs/stak/users isn't enough for the env and datasets
 - I needed to move my environment installation to /nfs/hpc/share
- Unfortunately, the HPC cluster is in a "bad place" so my env crashed
- I couldn't do any model training until last night, so I don't have any more results at this time...
 - We have the code, though!



11

11

Planned Results

Here's what we're going to do for the report:

- Train models using DP-SDG, SL, neither, and both
 - Neither is the "baseline" model
 - Will capture the accuracy results in wandb
 - Will capture the latency during training
- Attack each of the four models with all three methods
- Compare the results to the baseline
 - Relative success of the attacks
 - Accuracy (MRE) of the model
 - Latency of the training process

12

12

Implications / Conclusion



- We'll have some data-driven conclusions in the report...
- Reconstructing the dataset isn't *quite* enough to violate the privacy of the manufacturing process data
 - Reverse-engineering from simulation results is tricky
- Even so, nothing less than a *perfectly reconstructed* dataset would be useful since small variations matter
- A Private Federated Learning model (like what we covered in our paper presentation on Monday) is more appropriate for this problem, but not necessary for this project...

13

13



Oregon State
University

COLLEGE OF ENGINEERING

School of Electrical Engineering
and Computer Science



Any Questions

Matthew Jones, Aaryan Raghuvanshi, & Ren Yanghui

3 June 2026

Dr. Sanghyun Hong

14